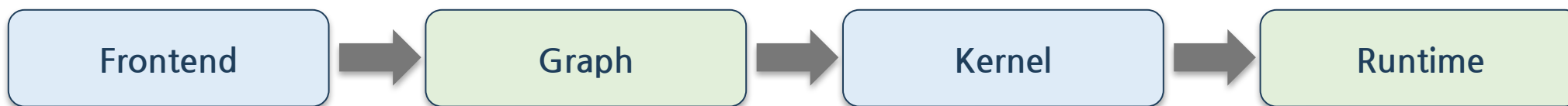


Lecture 05

Dialect System

Frontend / Graph / Kernel / Runtime Dialect 분리 설계



오늘의 위치

1-4 강에서 IR 구조와 tooling 을 배웠다 .

5 강부터는 MLIR 를 확장하는 방법을 다룬다 .

NPU compiler 에서는 dialect boundary 가 HW legality boundary 가 된다 .

Learning Objectives

Dialect 가 MLIR extension mechanism 에서 맡는 역할 설명
Operation/Type/Attribute/Trait/Interface/Verifier 의 관계 이해
frontend/graph/kernel/runtime dialect 분리 기준 수립
npu.matmul/npu.dma/npu.barrier pseudo op 설계
ODS/TableGen skeleton 읽기

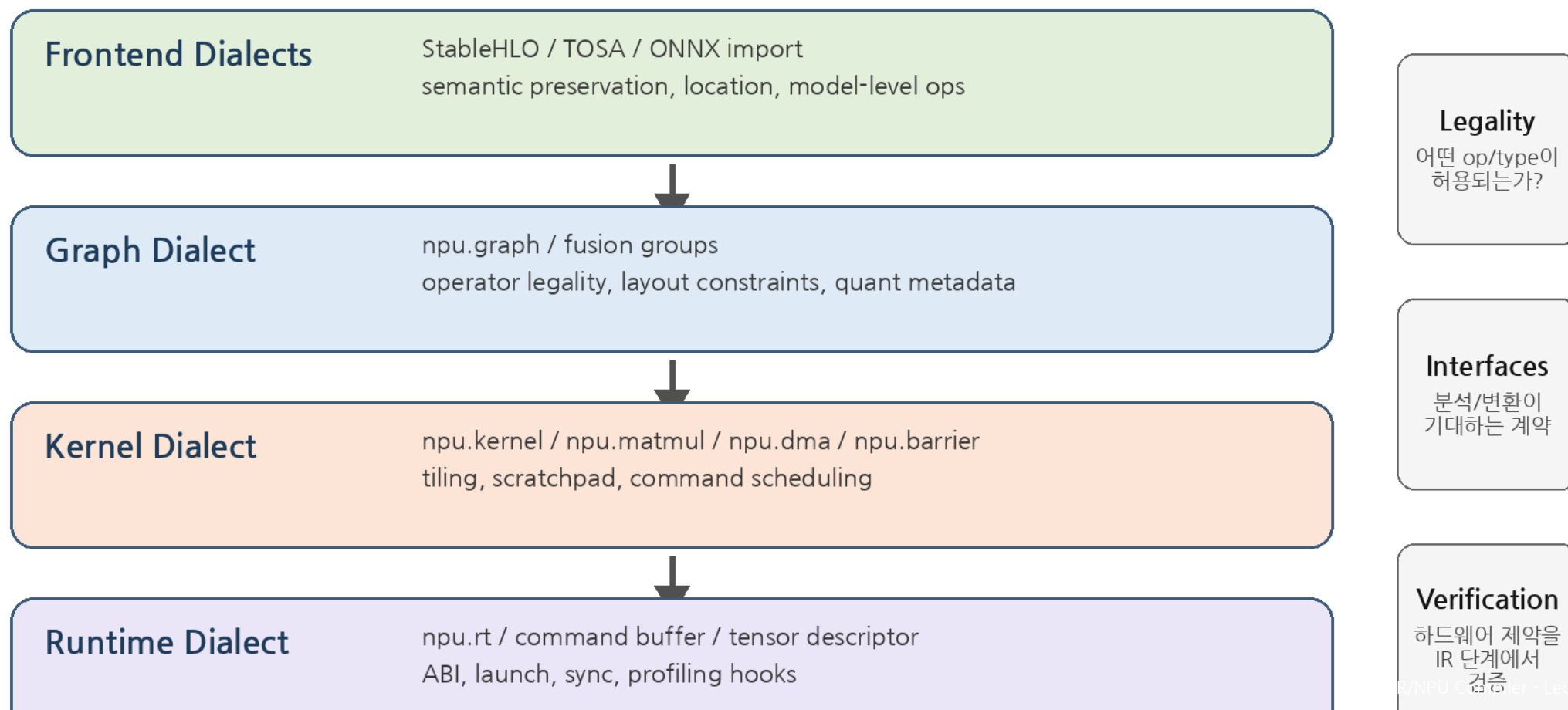
왜 Dialect 인가 ?

하나의 IR 에 model semantics 와 command buffer ABI 를 모두 넣으면 pass 가 복잡해진다 .
Dialect 는 abstraction level 별 semantic contract 를 만든다 .
Target-specific legality 를 조기에 검증할 수 있다 .
Out-of-tree NPU compiler 가 MLIR 생태계와 연결되는 접점이다 .

MLIR Dialect Layering

MLIR Dialect Layering for AI/NPU Compiler

각 dialect는 서로 다른 abstraction level과 correctness boundary를 갖는다.



Dialect = Namespace + Contract

operation 이름 : dialect.mnemonic

type 이름 : !dialect.type<...>

attribute 이름 : #dialect.attr<...>

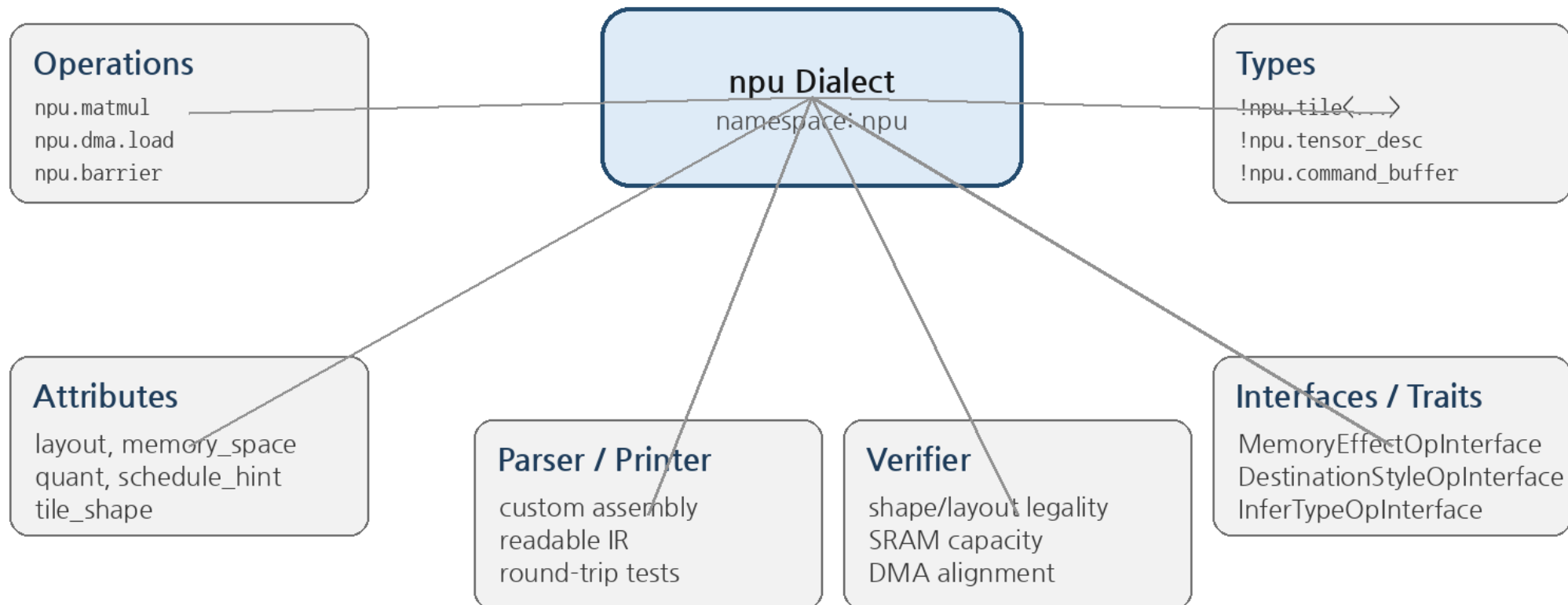
pass 와 verifier 는 이 contract 를 기준으로 동작

좋은 dialect 설계는 debug/test/runtime 까지 단순화한다 .

Dialect Anatomy

Dialect Anatomy

Dialect = namespace + operations + types + attributes + interfaces + verification + lowering boundary



Dialect 구성 요소

요소	역할	NPU 예시
Operation	IR 동작 / 계산 / 제어 표현	npu_kernel.matmul
Type	값의 shape/layout/ABI 계약	!npu_rt.tensor_desc
Attribute	compile-time metadata	tile_shape=[16,16,32]
Trait	구조적 성질 선언	Pure, SingleBlock
Interface	pass 가 호출하는 공통 semantic API	MemoryEffectOpInterface
Verifier	잘못된 target IR 거부	DMA alignment, SRAM capacity

Namespace Naming 원칙

이름만 봐도 abstraction level 이 보여야 한다 .
npu_graph.fusion_group: graph-level fusion
npu_kernel.matmul: tiled kernel compute
npu_rt.launch: runtime ABI launch
npu.op1 같은 이름은 test/debug 를 어렵게 한다 .

Frontend Dialect 책임

Framework semantic 보존

source location 과 debug metadata 유지

StableHLO/TOSA/ONNX import boundary

아직 NPU tile/DMA/detail 을 노출하지 않는다 .

목표 : 정확한 모델 의미와 portability

Graph Dialect 책임

Target-aware fusion group 표현
layout propagation / layout cast
quantization metadata 와 calibration binding
operator legality 와 partitioning
목표 : NPU 에 올릴 subgraph 를 명확히 한다 .

Kernel Dialect 책임

tile loop 와 compute primitive 표현
DMA load/store, barrier, async token
scratchpad SRAM 과 accumulator tile
array shape, tile shape, pipeline schedule
목표 : hardware scheduling legality 를 검증한다 .

Runtime Dialect 책임

tensor descriptor 와 command buffer

launch/sync/event/profiling

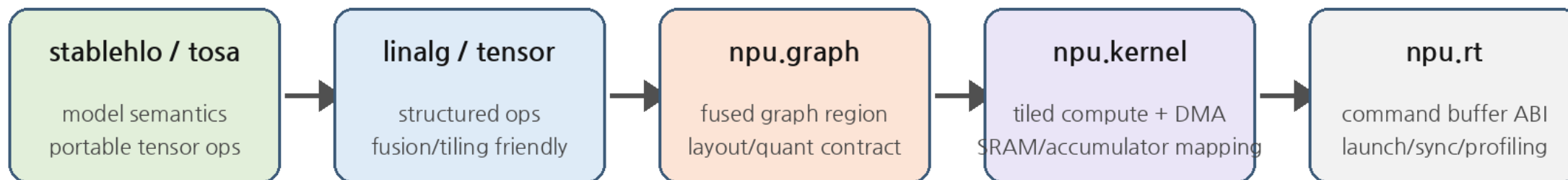
ABI version 과 descriptor layout

multi-core/core-mask/stream/QoS

목표 : driver/runtime 와 compiler 의 계약을 고정한다 .

Progressive Lowering Boundary

Dialect Boundary and Progressive Lowering



각 boundary에서 ConversionTarget, RewritePattern, TypeConverter, Verifier가 달라진다.

실무 원칙: HW 제약은 너무 늦게 runtime error로 발견하지 말고, 가능한 한 dialect verifier/pass에서 조기에 잡는다.

Attribute vs Operand vs Type

정보	권장 위치	판단 기준
Input tensor	Operand	dataflow/alias/lifetime 참여
tile_shape	Attribute	compile-time scheduling decision
SRAM memory space	Type 또는 Attribute	memory type 이면 Type, hint 면 Attribute
tensor descriptor	Type	runtime ABI 의미 포함
dynamic sequence length	Operand	runtime value
quant scale	Attribute 또는 Symbol	static metadata 또는 side-table

ODS/TableGen Skeleton

```
def NPUKernel_Dialect : Dialect {  
  let name = "npu_kernel";  
  let cppNamespace = "::npu::kernel";  
  let summary = "Kernel-level NPU operations";  
}  
  
class NPUKernel_Op<string mnemonic, list<Trait> traits = []>  
  : Op<NPUKernel_Dialect, mnemonic, traits>;
```


npu_kernel.matmul Pseudo ODS

```
def NPUKernel_MatMulOp : NPUKernel_Op<"matmul", []> {  
  let arguments = (ins AnyType:$lhs, AnyType:$rhs,  
                    I64ArrayAttr:$tile_shape,  
                    StrAttr:$accumulator_type);  
  let results = (outs AnyType:$acc);  
  let hasVerifier = 1;  
  
  // verifier: K match, tile legality, SRAM footprint,  
  // dtype/accumulator compatibility, array mapping  
}
```

Verifier 는 Target Contract 다

잘못된 IR 을 runtime 까지 보내지 않는다 .

shape/layout/dtype legality 를 조기에 검증

SRAM capacity, DMA alignment, barrier dependency 확인

FileCheck negative test 와 함께 관리

Verifier rule 은 hardware spec 과 같이 versioning 되어야 한다 .

Traits 와 Interfaces

Trait: operation 의 정적인 구조 / 성질 선언

Interface: pass 가 공통으로 호출할 semantic API

NPU DMA op 는 side effect 를 명확히 표현해야 한다 .

Tiling/fusion/bufferization pass 가 interface 를 통해 dialect 독립적으로 작동할 수 있다 .

Parser / Printer 가 중요한 이유

읽기 쉬운 IR 은 debug 속도를 결정한다 .
FileCheck 테스트가 안정적이고 짧아진다 .
custom assembly format 은 너무 마법적이면 안 된다 .
round-trip parse/print test 를 반드시 둔다 .

Pass 와 Dependent Dialects

pass 가 새 op/type/attribute 를 만들면 dialect loading 이 필요하다 .
getDependentDialects 누락은 out-of-tree build 와 멀티스레드 pipeline 에서 문제를 만든다 .
pass boundary 는 dialect boundary 와 함께 설계한다 .

Pass Dependency Pseudo C++

```
void getDependentDialects(DialectRegistry &registry) const override {  
    registry.insert<npu::kernel::NPUKernelDialect,  
                    memref::MemRefDialect,  
                    scf::SCFDialect,  
                    arith::ArithDialect>();  
}  
  
void runOnOperation() override {  
    // ConversionTarget + RewritePatternSet + TypeConverter  
}
```

Dialect Conversion Boundary

ConversionTarget: 어떤 op/dialect 가 legal 인가 ?

RewritePattern: illegal op 를 legal op 로 낮춘다 .

TypeConverter: tensor/memref/descriptor type 변환

Partial lowering 을 통해 여러 dialect 가 잠시 공존할 수 있다 .

Good Dialect Layering

Frontend semantic 을 너무 빨리 잃지 않는다 .

linalg/vector/scf/memref 최적화를 최대한 재사용한다 .

target-specific decision 이 필요할 때 custom NPU op 를 도입한다 .

runtime ABI 는 가장 낮은 level 에 둔다 .

Bad Smells

npu dialect 하나에 모든 op를 넣는다 .
operation 이름이 abstraction level을 설명하지 않는다 .
attribute에 중요한 semantic을 숨긴다 .
verifier 없이 pseudo op만 늘린다 .
custom op 도입이 너무 빨라 공통 최적화를 잃는다 .

Exercise A: Layer Classification

stablehlo.dot_general, layout hint, DMA load, tensor descriptor 를 분류
Frontend / Graph / Kernel / Runtime 중 하나 선택
각 항목의 pass boundary 와 verifier 책임 작성

Exercise B: npu_kernel.matmul 설계

operands/results/attributes/types 정의

traits/interfaces 후보 작성

verifier rule 최소 5 개 작성

assembly format 초안 작성

FileCheck test 이름 설계

Exercise C: Dialect Boundary Matrix

Import -> Graph

Graph -> Kernel

Kernel -> Runtime

Runtime -> Binary/Command

각 boundary 의 input/output dialect 와 legality check 작성

Takeaways

Dialect 는 MLIR 확장성의 핵심 단위다 .

NPU compiler 에서는 dialect boundary 가 hardware legality boundary 다 .

custom NPU dialect 는 standard dialect 최적화 재사용 이후에 도입하는 것이 좋다 .

ODS, verifier, interface, tests 까지 함께 설계해야 실전 compiler 가 된다 .

References

MLIR Language Reference

Defining Dialects

Creating a Dialect Tutorial

Operation Definition Specification

Defining Dialect Attributes and Types

Dialect Conversion

Pass Management